Red-black trees solutions

13-E01 Black-height and height conventions

Derivation and check

With red children, bh(20)=1, bh(10)=bh(30)=1, bh(NIL)=0, and h=2. Each path including root has2 blacks,20 and terminal NIL. With black children, bh(20)=2, child bh remains1, NIL remains0, h remains2, and complete count is3.

The parent's bh excludes the parent but includes a black child; the child's own bh excludes that child. Hence a black child has $bh(child) = bh(parent) - 1$, while a red child has equal bh. Height induction gives two subtrees of at least $2^{bh(x)-1} - 1$ internal nodes each, totaling $2^{bh(x)} - 1$. On a root-NIL path with B black and R red nodes, root/NIL black and nonadjacent reds imply R≤B−1. Thus h+1=B+R≤2B−1 and h≤2bh(root). Therefore $n \geq 2^{h/2} - 1$ and $h \leq 2\log_2(n+1)$.

The same keys and shape admit different valid black heights. At least roughly half the path nodes are black. Count every terminal NIL, not just internal black nodes.

13-E02 Old parent and beta pointers

Derivation and check

Save p30,y20,β15.
Write10.right=15,15.parent=10,20.parent=30,30.left=20,20.left=10,10.parent=20. External subtrees10.left and20.right remain. At the root, use20.parent=NIL and root=20 instead of30.left; other writes remain.

y's old parent is x. The external incoming link to change is x's old parent p pointing to y. Using y's old parent x can create a self-link or fail to reconnect p. Shared NIL.parent holds the current deletion x-position context. Writing it for another empty β can cause later sibling lookup at the wrong parent, so update β.parent only if β≠NIL.

Inorder remains α,10,15,20,γ; record data does not move. Check reverse parent links of all real children; the new root's parent is NIL.

13-E03 Two rotations for inner insertion

Derivation and check

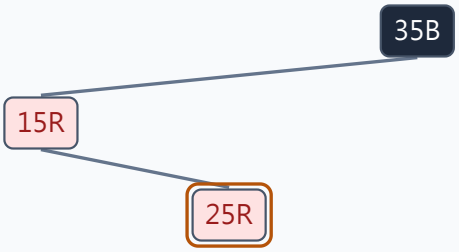
Insert35B, then35B-left15R. Inserting25 gives15.right=25R, a15R-25R conflict with black NIL uncle. I2 sets z=old parent15 and left-rotates15, yielding35B-left25R-left15R. z now denotes15, not inserted record25. I3 makes25 black and35 red and right-rotates35, yielding25B(left15R,right35R).

Red-black trees solutions (continued)

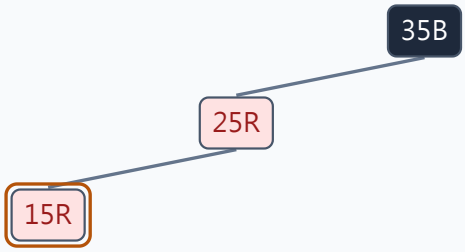
I2: left-rotate15 to make the inner chain outer

B=black, R=red; edges: children; orange: repair position; omitted children: black NIL.

Before: z=25, uncle NIL B



After: z=15, still red-red

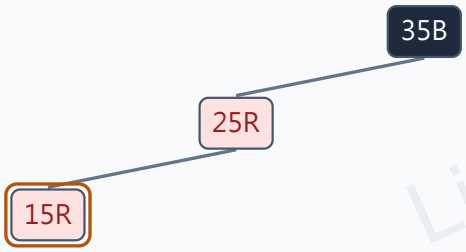


35 remains black root; inorder15,25,35. I3 must follow.

I3: blacken25, redden35, then right-rotate35

B=black, R=red; edges: children; orange: repair position; omitted children: black NIL.

Before I3: 25R / 15R



After I3: 25B, red children



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every root-NIL path has two blacks; black local root cannot conflict with its parent.

Generally I2 rotates two red internal nodes, each contributing zero black, and preserves black counts to boundary subtrees. In I3, let four black-rooted boundary subtrees contribute b each. Old gB paths and new pB paths both contribute1+b. New gR has black children and new zR has black parent, eliminating the sole conflict. This does not depend on the three-key example's b=1.

Rotation centers are15 then35, at most2 rotations. I2 alone retains a red conflict and must proceed to I3. Final inorder is15,25,35 and bh(root)=1.

Red-black trees solutions (continued)

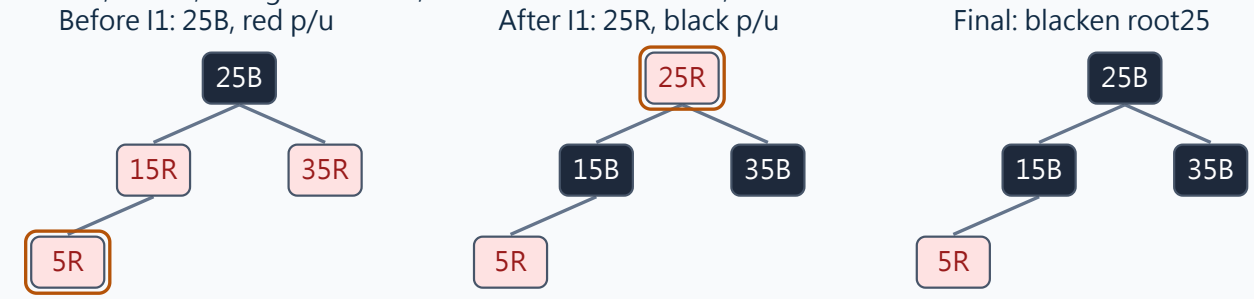
13-E04 Red uncle: complete counts versus bh

Derivation and check

Before I1, g25B,p15R,u35R,z5R. Every path including g has2 blacks and bh(g)=1. After I1,15/35 are black and25 red, with algorithm z=25. Complete count remains2, but bh(g)=2 because it excludes red g and includes one black child plus NIL. Final root blackening raises complete count to3 while bh(g) remains2.

Check I1 and root blackening separately

B=black, R=red; orange: current z; final root has no z mark; omitted children: black NIL.



Complete count2→2→3; bh(25)1→2→2. Nonroot g uses only the conserving middle step.

For nonroot g, its black-to-red loss cancels the red-to-black gain at p/u. Complete ancestor paths through g do not change. The only new conflict can be gR with g.parentR; z=g moves that loop problem up two levels. Other paths and BST order remain unchanged.

A larger bh(g) does not mean every whole-tree path gains a black. Only final root blackening produces that uniform increase.

13-E05 Preserve data while moving a successor

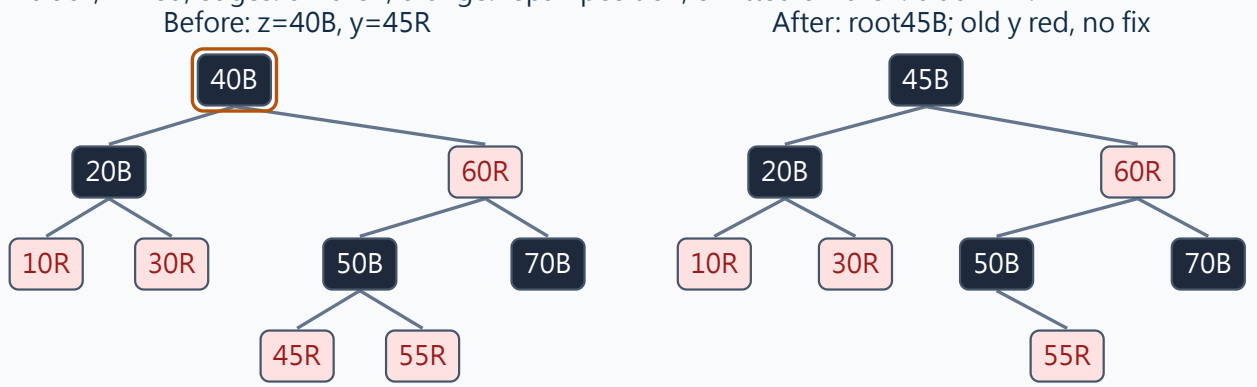
Derivation and check

z40B; successor y45R is50.left; x=y.right=NIL; save yOriginalColor=R. TRANSPLANT(y,NIL) gives50.left=NIL and NIL.parent=50. Set y.right=60 and60.parent=45. TRANSPLANT(40,45) sets root=45 and45.parent=NIL; attach45.left=20,20.parent=45; give45 the original black color of40.

Red-black trees solutions (continued)

Delete40: move the whole successor45 record to root

B=black, R=red; edges: children; orange: repair position; omitted children: black NIL.



D45 follows45; D40 leaves with40. Only key40 is lost; other identities remain.

Finally root45B has left20B(10R,30R), right60R with left50B(right55R), and right70B. y becomes black to preserve z's old-position count. Deficit depends on its original red color, so no fix runs. D45 remains in moved object45; D40 is removed with40. All other identities remain.

NIL.parent=50 describes this splice context and may be reset afterward. It does not describe every empty child's parent.

13-E06 Red sibling followed by red-parent absorption

Derivation and check

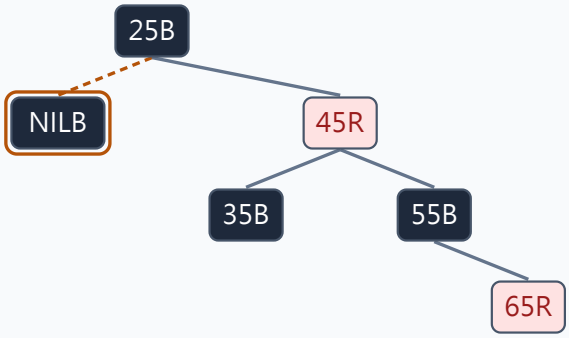
Before deletion, root25B has left15B and right45R(left35B,right55B(right65R)). Splice leaves x=NIL at25.left, parent25, sibling45R. D1 makes45 black and25 red, then left-rotates25. Root45B now has left25R(right35B) and right55B(right65R); x remains25.left and new w=35B.

Red-black trees solutions (continued)

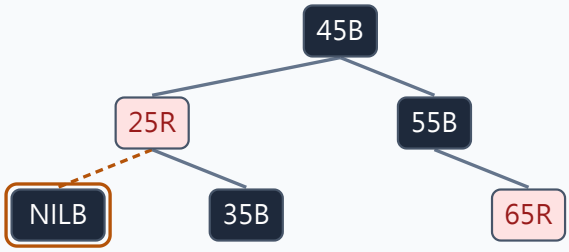
D1: blacken red sibling45 and left-rotate25

B=black, R=red; edges: children; orange: repair position; omitted children: black NIL.

Before: x=NIL, parent25



After: same x, sibling35B



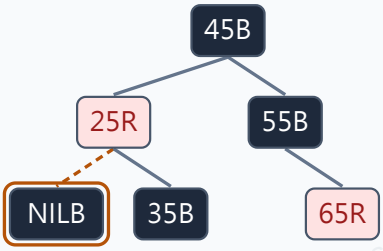
Orange NIL carries extra. It remains after conversion; proceed to D2 or D3/D4.

D2 makes35 red and x=25R. The loop exits; final blackening gives45B-left25B(right35R), right55B(right65R).

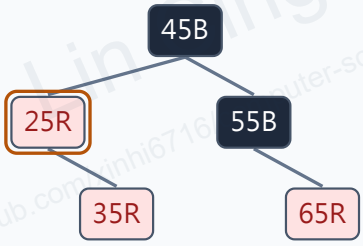
D2 and final blackening: move extra, then absorb it

B=black, R=red; orange border: x with extra; omitted children are black NIL.

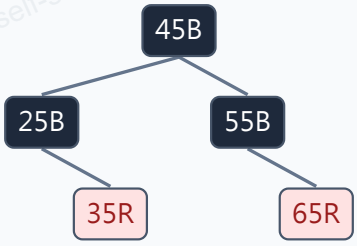
After D1: x=NIL + extra



After D2: x=25R + extra



Final: 25B absorbs extra



D2 makes35 red and moves x to red25. Final blackening of25 absorbs extra.

Let x's actual count be b and effective count b+1. Before D1, p black and w red with child counts b+1 give total b+2 including p. Afterward new w black has red p with effective branches b+1, and its other branch also b+1, retaining b+2. D2 makes p's actual branch counts both b and moves extra to p. D1 made p red, so blackening absorbs it without another upward iteration.

Only one rotation, at25, occurs. No second D1 is possible. Keep the25.left parent context even though NIL is shared.

Red-black trees solutions (continued)

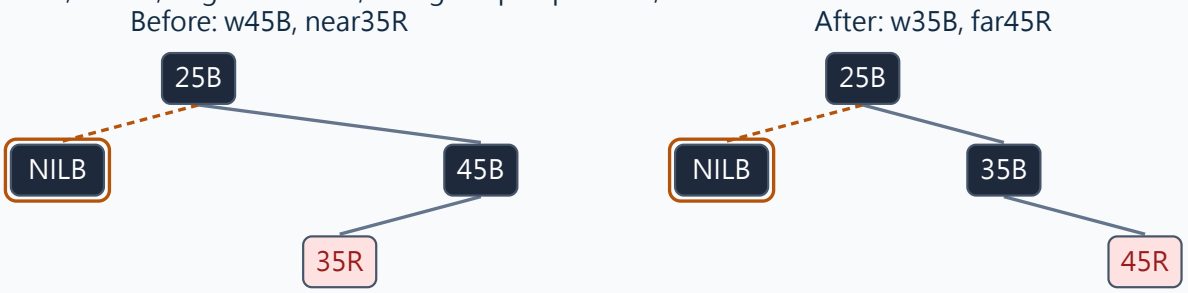
13-E07 Near-red, far-black must proceed to D4

Derivation and check

Before deletion,25B has left15B and right45B(left35R). Splice gives p25B,x=NIL at left,w45B,near35R,far=NIL black. D3 makes35 black and45 red, then right-rotates45, giving25B-right35B(right45R). x remains25.left, new w35B and far45R.

D3: near35 is red; right-rotate45 first

B=black, R=red; edges: children; orange: repair position; omitted children: black NIL.



x=NIL remains25.left. Extra remains; D4 must follow.

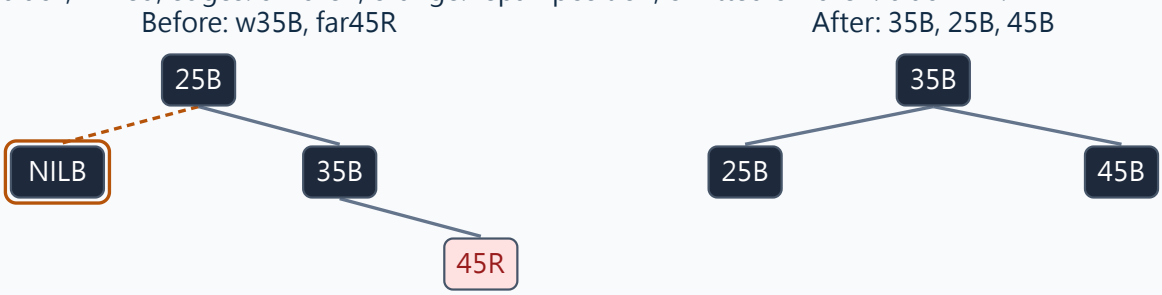
D4 gives35 the old black color of25, makes25/45 black, and left-rotates25. Result35B(left25B,right45B) has no extra.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black trees solutions (continued)

D4: blacken far45 and left-rotate25

B=black, R=red; edges: children; orange: repair position; omitted children: black NIL.



Keys25,35,45: every root-NIL path has three blacks; no extra remains.

Generally original effective local count is c+b+1. New w contributes c. Its left pB has children of actual count b, hence complete count b+1; red-to-black far also contributes b+1. New total c+b+1 matches. If w is red, both children are black, and its external color equals original p's color, preserving ancestor relationships.

D3 does not remove extra and cannot return. Rotation centers are45 then25. Final inorder25,35,45 has3 blacks on every complete root-NIL path.

13-E08 Black-parent D2 and deletion of the last node

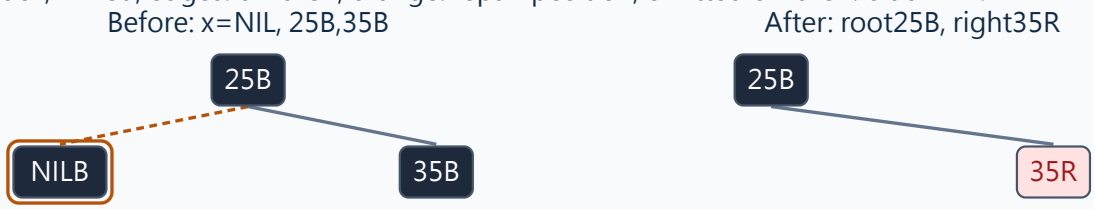
Derivation and check

Initially bh(root25)=2. Deleting15 leaves x=NIL at25.left and black w35 with black children. D2 makes35 red and moves x to root25B. The loop stops at the root and drops extra from all paths together. Result25B-right35R has bh(root)=1 with equal path counts.

Red-black trees solutions (continued)

D2 at root: whole-tree black height may drop by one

B=black, R=red; edges: children; orange: repair position; omitted children: black NIL.



x moves to root25; extra leaves every root-leaf path together. bh(root)2→1.

For singleton25B, x=25.right=NIL. TRANSPLANT sets root=NIL and NIL.parent=old25.parent=NIL. Original y was black, so fix is called, but x=root and the loop does nothing. Final x.color=B keeps NIL black. The empty key set has bh(NIL)=0,h=0 and all five properties.

Deletion may lower the whole-tree black height by one. Preserving equal counts does not require their numerical value to stay unchanged throughout the operation. The empty-tree case still requires black NIL.

13-E09 Right deficit and direct successor

Derivation and check

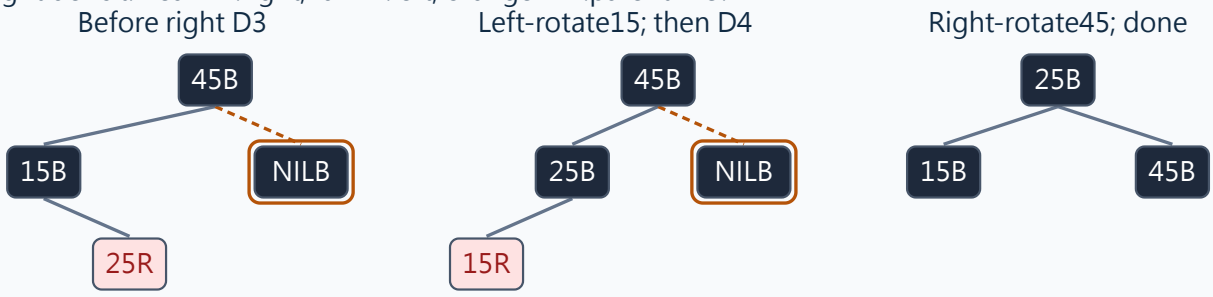
After insertion, root35B has left15B(right25R) and right45B. Deleting35 chooses direct successor y45B=z.right and x=NIL. Once y moves to root, x is45.right, so x.parent must be45, not old35.

w15B is left of x; near=w.right=25R and far=w.left=NIL black. Right-side D3 makes25 black and15 red, left-rotates15, and gives w25B,far15R. Right-side D4 gives25 the old black color of45, makes45/15 black, and right-rotates45. Final25B has15B,45B children.

Red-black trees solutions (continued)

Right D3/D4: mirror directions, near, and far

Right deficit: near=w.right, far=w.left; orange NIL.parent=45.



Direct successor45B replaces root35; final25B(left15B,right45B) retains record45.

Whole record45 moves from successor position to local root, then becomes25.right through rotation. D45 remains;35 and D35 are removed. Mirroring only sibling would misidentify w.left=NIL as the red near child or attempt a right rotation at15 with no left child, violating the rotation premise.

Mirror child tests, near/far, rotation directions, and parent wiring together.

13-E10 Height bounds versus rotation bounds

Derivation and check

bh(root)=3 gives $n \geq 2^3-1=7$ and directly $h \leq 6$. Knowing only $n \geq 7$ does not permit substituting n=7 into a general upper bound for any larger tree. At n=1023, $h \leq 2\log_2 1024=20$. Search takes O(h); complete traversal outputs1023 nodes and takes $\Theta(n)$.

Only I1 repeats during insertion, without rotation. I2 proceeds to terminating I3, for at most2 rotations. Only D2 repeats upward in deletion, without rotation. D1 makes p red; D2 then exits with red x, or D3/D4 immediately terminates. Thus D1 occurs at most once. D3/D4 each occur at most once, totaling at most3 rotations.

I1 or D2 can still recolor/count along $\Theta(\log n)$ ancestors, and the initial search can require that height. Constant rotation counts do not imply worst-case constant total time.

Red-black trees solutions (continued)

13-E11 Sentinel and API invariants

Derivation and check

Check (a),(c),(d),(e). Statement(b) cannot hold simultaneously at all empty positions because one NIL.parent is shared. During deletion it can hold the current x-position parent, and rotations must not overwrite it for another empty β. It may be reset to NIL in a stable tree; NIL.color is always black.

Duplicate insertion finds the equal key during search and returns that same record, without a new red leaf or shape/color change. Deleting an absent key returns NOT_FOUND without RB-DELETE. NIL has no defined key. Treating it as ordinary z loses valid-position and identity premises and can corrupt root or sentinel-parent wiring.

Require reverse parent consistency for every real internal child. NIL is a contextual exception, not a reason to omit ordinary parent checks.

13-E12 Rotation alone does not preserve red-black properties

Derivation and check

After left-rotating20, root30R has left20B, whose left child is10R. Inorder remains10,20,30. A red root violates property2. The path from30 directly to right NIL has one black, terminal NIL. Paths through20.right NIL or through10's NIL contain20 and NIL, totaling2. Property5 also fails.

Blackening only root30 changes the right count to2 and left counts to3, still unequal. BST order constrains key intervals and child ordering, not colors or black height. Thus preserving BST order is fully consistent with failing red-black properties.

A legal repair must check its case's color premises and black-count conservation. Arbitrary rotation followed by root blackening is insufficient.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Red-black trees solutions (continued)

Verification code appendix: red_black_checks.py

```
"""Red-black operation behavior, branch witnesses and stable record identities."""
from itertools import permutations
from random import Random
from math import log2


class Node:
    def __init__(self,key=None,color='B'):
        self.key=key;self.color=color;self.payload=('record',key)
        self.left=self.right=self.parent=None


class Tree:
    def __init__(self,capture=False):
        self.nil=Node();self.nil.left=self.nil.right=self.nil.parent=self.nil
        self.root=self.nil;self.capture=capture;self.events=[];self.rotations=0
    def snapshot(self):
        def visit(x):return None if x is self.nil else
(x.key,x.color,visit(x.left),visit(x.right))
        return visit(self.root)
    def event(self,code,x):
        if self.capture:self.events.append({'code':code,'tree':self.snapshot(),'focus':None
if x is self.nil else x.key,'parent':None if x.parent is self.nil else x.parent.key})
        else:self.events.append(code)
    def left_rotate(self,x):
        y=x.right;assert y is not self.nil;p=x.parent
        x.right=y.left
        if y.left is not self.nil:y.left.parent=x
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Red-black trees solutions (continued)

```
        y.parent=p
        if p is self.nil:self.root=y
        elif x is p.left:p.left=y
        else:p.right=y
        y.left=x;x.parent=y;self.rotations+=1
    def right_rotate(self,x):
        y=x.left;assert y is not self.nil;p=x.parent
        x.left=y.right
        if y.right is not self.nil:y.right.parent=x
        y.parent=p
        if p is self.nil:self.root=y
        elif x is p.right:p.right=y
        else:p.left=y
        y.right=x;x.parent=y;self.rotations+=1
    def search(self,key):
        x=self.root
        while x is not self.nil and x.key!=key:x=x.left if key<x.key else x.right
        return x
    def insert(self,key):
        self.events=[];self.rotations=0;p=self.nil;x=self.root
        while x is not self.nil:
            p=x
            if key==x.key:return x
            x=x.left if key<x.key else x.right
        z=Node(key,'R');z.left=z.right=self.nil;z.parent=p
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black trees solutions (continued)

```
        if p is self.nil:self.root=z
        elif key<p.key:p.left=z
        else:p.right=z
        original=z;self.event('insert:leaf',z)
        while z.parent.color=='R':
            p=z.parent;g=p.parent;left=p is g.left;side='L' if left else 'R'
            uncle=g.right if left else g.left
            if uncle.color=='R':
                self.event('I-'+side+'1:before',z)
                p.color=uncle.color='B';g.color='R';z=g
                self.event('I-'+side+'1:after',z)
            else:
                if (z is p.right) if left else (z is p.left):
                    self.event('I-'+side+'2:before',z)
                    z=p
                    (self.left_rotate if left else self.right_rotate)(z)
                    self.event('I-'+side+'2:after',z)
                self.event('I-'+side+'3:before',z)
                z.parent.color='B';z.parent.parent.color='R'
                (self.right_rotate if left else self.left_rotate)(z.parent.parent)
                self.event('I-'+side+'3:after',z)
        self.root.color='B';self.root.parent=self.nil;self.nil.parent=self.nil
        self.event('insert:end',original);assert self.rotations<=2
        return original
    def transplant(self,u,v):
        if u.parent is self.nil:self.root=v
        elif u is u.parent.left:u.parent.left=v
        else:u.parent.right=v
        v.parent=u.parent
    def minimum(self,x):
        while x.left is not self.nil:x=x.left
        return x
    def delete(self,key):
        self.events=[];self.rotations=0;z=self.search(key)
        if z is self.nil:return False
        self.event('delete:start',z);y=z;color=y.color
        if z.left is self.nil:
            x=z.right;self.transplant(z,x)
        elif z.right is self.nil:
            x=z.left;self.transplant(z,x)
        else:
            y=self.minimum(z.right);color=y.color;x=y.right
            if y.parent is z:x.parent=y
            else:
                self.transplant(y,x);y.right=z.right;y.right.parent=y
            self.transplant(z,y);y.left=z.left;y.left.parent=y;y.color=z.color
        self.event('delete:splice',x)
        if color=='B':self.fix_delete(x)
        if self.root is not self.nil:self.root.parent=self.nil
        self.nil.parent=self.nil;z.left=z.right=z.parent=self.nil
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Red-black trees solutions (continued)

```
        self.event('delete:end',self.root);assert self.rotations<=3
        return True
    def fix_delete(self,x):
        while x is not self.root and x.color=='B':
            p=x.parent;left=x is p.left;side='L' if left else 'R'
            w=p.right if left else p.left;assert w is not self.nil
            if w.color=='R':
                self.event('D-'+side+'1:before',x)
                w.color='B';p.color='R'
                (self.left_rotate if left else self.right_rotate)(p)
                w=x.parent.right if left else x.parent.left
                self.event('D-'+side+'1:after',x)
            near=w.left if left else w.right;far=w.right if left else w.left
            if near.color==far.color=='B':
                self.event('D-'+side+'2:before',x)
                w.color='R';x=x.parent
                self.event('D-'+side+'2:after',x)
            else:
                if far.color=='B':
                    self.event('D-'+side+'3:before',x)
                    near.color='B';w.color='R'
                    (self.right_rotate if left else self.left_rotate)(w)
                    w=x.parent.right if left else x.parent.left
                    self.event('D-'+side+'3:after',x)
                self.event('D-'+side+'4:before',x)
                w.color=x.parent.color;x.parent.color='B'
                (w.right if left else w.left).color='B'
                (self.left_rotate if left else self.right_rotate)(x.parent)
                x=self.root
                self.event('D-'+side+'4:after',x)
        x.color='B'
    def check(self,expected):
        assert self.nil.color=='B' and self.nil.left is self.nil and self.nil.right is
self.nil
        assert self.root.parent is self.nil and self.root.color=='B'
        seen={};keys=[]
        def visit(x,p,lo,hi):
            if x is self.nil:return 1,0
            assert id(x) not in seen;seen[id(x)]=x
            assert x.parent is p and lo<x.key<hi and x.color in ('R','B')
            assert x.payload==('record',x.key)
            if x.color=='R':assert x.left.color==x.right.color=='B'
            lb,lh=visit(x.left,x,lo,x.key);keys.append(x.key);rb,rh=visit(x.right,x,x.key,hi)
            assert lb==rb
            return lb+(x.color=='B'),1+max(lh,rh)
        bh,height=visit(self.root,self.nil,float('-inf'),float('inf'))
        assert keys==sorted(expected) and height<=2*log2(len(expected)+1)
        return {x.key:x for x in seen.values()},bh-1 if self.root is not self.nil else
0,height


def build(order,capture=False):
    t=Tree(capture)
```

Red-black trees solutions (continued)

```
    for key in order:t.insert(key)
    return t


def main():
    witnesses={};insertions=deletions=0;coverage=set()
    def record(t,order,deleted=None):
        for code in t.events:
            if ':before' in code:
                k=code.split(':')[0];coverage.add(k)
                witnesses.setdefault(k,{'insert':list(order),'delete':deleted})
    for n in range(1,7):
        for order in permutations(range(1,n+1)):
            t=Tree();expected=set()
            for i,key in enumerate(order):

t.insert(key);expected.add(key);t.check(expected);record(t,order[:i+1]);insertions+=1
            for key in order:
                q=build(order);live=q.check(expected)[0];assert q.delete(key)
                now=q.check(expected-{key})[0]
                assert all(now[k] is node for k,node in live.items() if k!=key)
                record(q,order,key);deletions+=1
    rng=Random(13024);t=Tree();expected=set();mixed=0
    for _ in range(6000):
        key=rng.randrange(100)
        if rng.random()<.55:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black trees solutions (continued)

```
            before=t.search(key);node=t.insert(key)
            if before is not t.nil:assert node is before
            expected.add(key)
        else:
            old=key in expected;assert t.delete(key)==old;expected.discard(key)
        t.check(expected);mixed+=1
        coverage.update(code.split(':')[0] for code in t.events if ':before' in code)
    for key in sorted(expected,reverse=True):assert t.delete(key)
    t.check(set());assert t.root is t.nil and not t.delete(999)
    wanted={f'I-{s}{k}' for s in 'LR' for k in [1,2,3]}|{f'D-{s}{k}' for s in 'LR' for k in
[1,2,3,4]}
    assert wanted<=coverage,(wanted-coverage)
    # Explicit NIL-parent and successor-moving object paths, including direct and deep
successor.
    for order,key in [([40,20,60,10,30,50,70,45,55],40),([20,10,30,5,15,35],20),([1],1)]:
        t=build(order);before=t.check(set(order))[0];t.delete(key);after=t.check(set(order)-
{key})[0]
        assert all(after[k] is node for k,node in before.items() if k!=key)
    print(f'PASS insertion_prefixes={insertions} single_deletions={deletions}
mixed_operations={mixed} '
          'all14_mirrored_fixup_cases=True
root_NIL_global_BST_parent_blackheight_redchildren_payload_identities=True '
          'max_insert_rotations2_max_delete_rotations3=True')
    print('WITNESSES',witnesses)
    return witnesses


if __name__=='__main__':main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study